

UNIDAD PROFESIONAL INTERDISCIPLINARIA
DE INGENIERÍA Y
CIENCIAS SOCIALES Y ADMINISTRATIVAS
(UPIICSA)

Practica 3 de Abstracción de datos

Secuencia: 3CM30

INTEGRANTES:

Arcos García Andreé

Hernandez Eleria Juan Jose

Mejia Sanchez Aldo

Repositorio de github

<https://github.com/Al-Mejia-0920/Programas-de-Segundo-Parcial>

Introducción

En el desarrollo de programas modernos, la gestión y persistencia de los datos son uno de los pilares fundamentales de la arquitectura de software. Generalmente los programas que se venían trabajando ocupaban únicamente la memoria RAM, sin embargo, al tratarse de una memoria volátil, los datos se pierden al finalizar la ejecución del programa.

En este tercer departamento, estudiamos e implementamos el trabajo con sistemas de archivos del sistema operativo. Se utilizaron los tres formatos de archivo de texto:

- **TXT (Texto Plano y Tabular/CSV):** Utilizado para el volcado rápido de información y reportes lineales masivos debido a su alta eficiencia en espacio y nulo consumo de procesamiento en parseo.
- **XML (Extensible Markup Language):** Enfocado en el almacenamiento jerárquico y estricto de configuraciones y catálogos maestros mediante estructuras de etiquetas anidadas en forma de árbol.
- **JSON (JavaScript Object Notation):** Empleado como el estándar actual de transferencia de datos por su ligereza, legibilidad humana y mapeo directo con la programación orientada a objetos (POO).

Así también trabajamos con las estructuras de grafos, digrafos y árboles para la lectura de archivos de los tres tipo de formatos ya antes mencionados, *a continuación, se detalla la implementación técnica de estas estructuras de datos, su código fuente y las pruebas de ejecución correspondientes.*

PROGRAMAS

Archivos :

La idea de este programa es convertir los programas del 1er parcial y del segundo parcial en 4 tipos de datos, los cuales son : TXT, JSON, XML y CSV.

Esto sirve para ahorrarte el trabajo manual a la hora de probar y documentar los trabajos. En lugar de compilar cada código a mano y "copiar y pegar" lo que sale en la pantalla negra de la consola, el programa centraliza todo el proceso y hace lo siguiente:

- Abre y corre automáticamente los ejercicios del 1er y 2do parcial.
- Guarda todo el texto que esos ejercicios imprimen durante su ejecución.
- Toma esos resultados crudos de la terminal y te permite convertirlos en cuatro formatos estructurados: TXT, JSON, XML y CSV.

```
-----
                        1ER PARCIAL
-----
Ejercicios: 1, 2, 3, 4, 5, 6, 7, 7.1, 8, 9, 10, 11, 12, 13, 14, 15
-----
Ingresa el numero del ejercicio que deseas ejecutar : 3

-----
                        PROGRAMA EN EJECUCION
-----

La transcripción ha comenzado. El archivo de salida es salida_temp.txt
Inserta la operacion que quieres realizar (1.- Suma, 2.-Resta, 3.-Multiplicacion, 4.-Division: 3
Inserta el primer numero a operar: 50
Inserta el segundo numero a operar: 30
La multiplicacion es: 1500
La transcripción se ha detenido. El archivo de salida es C:\Users\THERMALTAKE\Documents\Yventz\3erP
ARCIAL\salida_temp.txt

-----
                        SELECCIONA TU FORMATO
-----

1. JSON | 2. XML | 3. CSV | 4. TXT
Elige una opcion: |
```

```
-----
                        SELECCIONA TU FORMATO
-----

1. JSON | 2. XML | 3. CSV | 4. TXT
Elige una opcion: 1
Nombre del nuevo archivo :(sin extension): multiplicacion

¡Exito! Archivo multiplicacion.json creado.

Presiona ENTER para regresar al menu principal...|
```

```
{
  "archivo": "resultado_ejecucion",
  "lenguaje": "C++",
  "datos_ingresados": "Ejecucion completa",
  "lineas_de_salida": [
    {
      "linea": 1,
      "texto": "La transcripción ha comenzado. El archivo de salida es salida_temp.txt"
    },
    {
      "linea": 2,
      "texto": "Inserta la operacion que quieres realizar (1.- Suma, 2.-Resta, 3.-Multiplicacion, 4.-Division: 3"
    },
    {
      "linea": 3,
      "texto": "Inserta el primer numero a operar: 50"
    },
    {
      "linea": 4,
      "texto": "Inserta el segundo numero a operar: 30"
    },
    {
      "linea": 5,
      "texto": "La multiplicacion es: 1500"
    }
  ]
}
```

En resumen, facilita la creación de reportes o preparar esos datos para que puedan ser leídos por otras aplicaciones.

En general tuvimos muchas problemáticas las cuales las que más destacaron fueron las siguientes :

PROBLEMÁTICA

Nosotros ya teníamos nuestros códigos de las prácticas hechos (archivos **.cpp** sueltos) y no queríamos meternos a editar cada uno de ellos para agregarles funciones de crear archivos. Solo quería capturar lo que arrojaban en la consola negra.

```
if (procederaconversion) {
    cout << "\n-----" << endl;
    cout << "        PROGRAMA EN EJECUCION        " << endl;
    cout << "-----\n" << endl;

    system("powershell -Command \"Start-Transcript -Path 'salida_temp.txt' -Force; .\\programa_temp.exe; Stop-Transcript\"");

    ifstream archivo("salida_temp.txt");
    string linea;
    if (archivo.is_open()) {
        int contadorasteriscos = 0;
```

La solución fue que investigando, descubrimos que podíamos usar la terminal de Windows a nuestro favor . En la función **system()** de nuestro main , metimos un comando de PowerShell llamado **Start-Transcript**. Lo que hace este comando es grabar absolutamente todo lo que pasa en la consola mientras nuestros programas se ejecutan y

guardarlo temporalmente en un archivo llamado **salida_temp.txt**. Así logramos extraer la información sin tocar los códigos originales.

Otra problemática muy pesada fue cuando abrimos nuestro archivo **salida_temp.txt** por primera vez, vimos que no solo venían los resultados esperados, sino un montón de texto extra del sistema: encabezados de PowerShell, la ruta de las carpetas (PS C:\Users\...), tiempos de ejecución, etc. Si pasábamos eso directo a un JSON, llegaba con toda esta información.

```
while (getline(archivo, linea)) {
    if (linea.find("*****") != string::npos) {
        contadorasteriscos++;
        continue;
    }
    if (contadorasteriscos < 2 || contadorasteriscos >= 3) {
        continue;
    }
    if (linea.find("PS ") != string::npos && linea.find(">") != string::npos) continue;
    if (linea.find(".\\programa_temp.exe") != string::npos) continue;

    if (!linea.empty()) {
        contenido += linea + "\n";
    }
}
```

Para solucionarlo tuvimos que crear un "filtro". Nos dimos cuenta de que PowerShell siempre encierra sus encabezados entre líneas llenas de asteriscos (*****). Así que programamos un ciclo while que lee el archivo temporal línea por línea y lleva un contador. Le indicamos a nuestro programa: *"Ignora todo hasta que cuentes dos líneas de asteriscos, y también sáltate cualquier línea que tenga 'PS >' o el nombre del ejecutable"*. Con esto, logramos rescatar únicamente el texto puro de nuestras estructuras de datos.

Después notamos que si las prácticas imprimían comillas dobles (") o saltos de línea, la estructura del archivo JSON o CSV se rompía todo y el sistema marcaba error de sintaxis al intentar abrirlos

```
string prepararparajson(string texto) {
    string resultado = "";
    for (char letra : texto) {
        if (letra == '"') resultado += "\\\"";
        else if (letra == '\\') resultado += "\\\\";
        else if (letra == '\n') resultado += "\\n";
        else if (letra != '\r') resultado += letra;
    }
    return resultado;
}
```

Para esto nuestra solución fue que escribiéramos unas funciones de apoyo (**prepararparajson y prepararparacsv**). Lo que hacen es recorrer el texto letra por letra antes de guardarlo, Si encuentran una comilla, le ponen una diagonal invertida antes (\") o la duplican, dependiendo del formato que se haya elegido. Es un detalle pequeño pero fue clave para que los archivos generados fueran 100% funcionales y no se corrompieran

Otra de las problemáticas no tan grandes que teníamos era que teníamos que exportar los resultados a 4 formatos distintos (JSON, XML, CSV y TXT). Si intentábamos hacer todo eso dentro del **main**, íbamos a terminar con un bloque de código muy grande , confuso y lleno de condicionales **if/else**.

```
main.cpp X json.cpp X xml.cpp X txt.cpp X convertidor.h X csv.cpp X
1  #ifndef CONVERTIDOR_H
2  #define CONVERTIDOR_H
3  #include <string>
4  #include <fstream>
5
6  using namespace std;
7
8  class IConvertidor {
9  public:
10     virtual ~IConvertidor() {}
11     virtual string generartexto() = 0;
12     virtual void guardar(string nombrearchivo) = 0;
13 };
14
15 class Convertidor : public IConvertidor {
16 protected:
17     string contenido;
18     string datos;
19 public:
20     Convertidor(string cont, string dat) : contenido(cont), datos(dat) {}
21     virtual string generartexto() = 0;
22     void guardar(string nombrearchivo) override {
23         ofstream archivo(nombrearchivo);
24         archivo << generartexto();
25         archivo.close();
26     }
27 };
28
29 class ConvertidorTXT : public Convertidor {
30 public:
31     ConvertidorTXT(string cont, string dat) : Convertidor(cont, dat) {}
32     string generartexto() override;
33 };
34
```

Para esto decidimos utilizar Programación Orientada a Objetos como lo indicó el profesor, creamos una estructura basada en Herencia y Polimorfismo. Hicimos una clase base abstracta llamada **Convertidor** y de ahí clases hijas específicas para cada formato (como **ConvertidorJSON** o **ConvertidorXML**), gracias a esto el **main** quedo limpio y ya solo pregunta qué formato queremos, crea el objeto correspondiente y llama a una única función **guardar()**. Cada clase sabe cómo formatear su propio texto.

Grafo:

```
5     using namespace std;
6
7     int main() {
8         GrafoDinamico miMapa("red.xml");
9         miMapa.cargarConfiguracionXML();
10
11         char opcion;
12         cout << "RUTAS" << endl;
13
14         do {
15             Ruta nuevaRuta;
16
17             cout << "\nIngrese el Nodo de Origen (Ej. A-F): ";
18             cin >> nuevaRuta.n;
19             nuevaRuta.n = toupper(nuevaRuta.n);
20
21             cout << "Ingrese el Nodo de Destino (Ej. A-F): ";
22             cin >> nuevaRuta.nf;
23             nuevaRuta.nf = toupper(nuevaRuta.nf);
24
25             cout << "Ingrese el nombre de la Arista (Ej. a-n): ";
26             cin >> nuevaRuta.aC;
27
28             cout << "Ingrese el Peso/Costo de esta conexion: ";
```

```
Ingrese el nombre de la Arista (Ej. a-n): b
Ingrese el Peso/Costo de esta conexion: 10

¿Desea registrar otra ruta? (S/N): s

Ingrese el Nodo de Origen (Ej. A-F): c
Ingrese el Nodo de Destino (Ej. A-F): b
Ingrese el nombre de la Arista (Ej. a-n): c
Ingrese el Peso/Costo de esta conexion: 10

¿Desea registrar otra ruta? (S/N): s

Ingrese el Nodo de Origen (Ej. A-F): b
Ingrese el Nodo de Destino (Ej. A-F): f
Ingrese el nombre de la Arista (Ej. a-n): d
Ingrese el Peso/Costo de esta conexion: 10

¿Desea registrar otra ruta? (S/N): f
MAPA DEL GRAFO
Nodo [F] -> NULL
Nodo [E] -> NULL
Nodo [D] -> NULL
Nodo [C] -> --(c, peso:10)--> B | --(b, peso:10)--> C | NULL
Nodo [B] -> --(d, peso:10)--> F | NULL
Nodo [A] -> --(a, peso:10)--> C | NULL
CAMINO MAS CORTO
Ingrese desde que Nodo desea partir:
```

```

<?xml version="1.0" encoding="UTF-8"?>
<configuracion>
  <nodo id="A"/>
  <nodo id="B"/>
  <nodo id="C"/>
  <nodo id="D"/>
  <nodo id="E"/>
  <nodo id="F"/>

  <arista id="a"/>
  <arista id="b"/>
  <arista id="c"/>
  <arista id="d"/>
  <arista id="e"/>
  <arista id="f"/>
  <arista id="g"/>
  <arista id="h"/>
  <arista id="i"/>
  <arista id="j"/>
  <arista id="k"/>
  <arista id="l"/>
  <arista id="m"/>
  <arista id="n"/>
</configuracion>

```

Este primer ejercicio consistió en el desarrollo de un dígrafo dinámico orientado a la resolución de problemas de optimización de rutas. La arquitectura del software se diseñó de manera desacoplada utilizando un archivo externo en formato XML, el cual funge exclusivamente como el catálogo maestro del sistema al indicar cuáles nodos y aristas son válidos y están autorizados dentro del universo del problema.

A partir de esta configuración inicial, el programa interactúa directamente con el usuario por medio de la consola. En lugar de contar con un grafo rígido, somos nosotros quienes vamos indicando dinámicamente las rutas, seleccionando qué nodos específicos se van a conectar, a través de qué aristas del catálogo y asignándoles su respectivo peso o costo en tiempo de ejecución

Grafo.cpp X main.cpp X grafo.h X red.xml X

Primer problema de desarrollo: Al inicio el código no se trabajó como un proyecto dentro del IDE, sino con archivos sueltos. Esto provocó que el enlazador no asociara la lógica de grafo.cpp con el main.cpp, arrojando el error de referencias indefinidas ld returned 1 exit status. Se solucionó migrando el código a un proyecto formal de Aplicación de Consola en Code Blocks para vincular todos los archivos de manera automática.

Segundo problema de desarrollo: El validador del sistema marcaba error si el usuario escribía un nodo en minúscula, por ejemplo una letra a en lugar de una letra A, debido a la diferencia de valores en la tabla ASCII respecto al catálogo configurado en el XML. Se solucionó incluyendo la librería ctype y aplicando la función toupper inmediatamente después de capturar los datos con cin para

estandarizar todas las entradas a mayúsculas automáticamente antes de ser evaluadas.

Dígrafo:

```
#ifndef DIGRAFO_H
#define DIGRAFO_H

#include <string>
#include <vector>

struct Ruta {
    char n;
    char nf;
    std::string aC;
    int p;
};

struct Arista {
    char destino;
    std::string nombreArista;
    int peso;
    Arista* siguiente;
};

struct Vertice {
    char id;
    Arista* listaAristas;
    Vertice* siguiente;
};
```

```

Ingrese el Nodo de Origen (Ej. A-F): a
Ingrese el Nodo de Destino (Ej. A-F): f
Ingrese el nombre de la Arista (Ej. a-n): e
Ingrese el Peso/Costo de esta conexion: 10

¿Desea registrar otra ruta? (S/N): s

Ingrese el Nodo de Origen (Ej. A-F): f
Ingrese el Nodo de Destino (Ej. A-F): e
Ingrese el nombre de la Arista (Ej. a-n): f
Ingrese el Peso/Costo de esta conexion: 10

¿Desea registrar otra ruta? (S/N): e
      MAPA DEL GRAFO
Nodo [F] -> --(f, peso:10)--> E | NULL
Nodo [E] -> NULL
Nodo [D] -> NULL
Nodo [C] -> --(d, peso:10)--> C | --(c, peso:10)--> B | NULL
Nodo [B] -> NULL
Nodo [A] -> --(e, peso:10)--> F | --(b, peso:10)--> C | --(a, peso:10)--> B | NULL
      CAMINO MAS CORTO
Ingrese desde que Nodo desea partir: a
Ingrese hacia que Nodo desea llegar: f

[Dijkstra] Ruta mas corta desde A hasta F | Peso Total: 10
A --(e)--> F

Process returned 0 (0x0)    execution time : 84.476 s
Press any key to continue.

```

En este caso no fue mucho problema el dígrafo, puesto que a la hora de hacerlo, teníamos como base el programa de grafos y solo tuvimos que darle dirección para tener nuestro digrafo

Árbol:

Utilizamos un Árbol Binario de Búsqueda. Su característica principal es que cada nodo se divide en un máximo de dos subárboles bajo una regla matemática: los valores menores al nodo actual se almacenan siempre en el lado izquierdo, y los valores mayores se posicionan en el lado derecho.

Arboles.h: Este archivo funciona como la base de nuestro programa. En él definimos la estructura `NodoArbol`, la cual utiliza dos punteros para establecer las conexiones del árbol binario: uno para el subárbol izquierdo (`izq`) y otro para el subárbol derecho (`der`). Asimismo, este archivo actúa como un plano general donde declaramos todas las acciones y métodos que nuestro árbol puede realizar, indicando qué operaciones existen, pero sin programar todavía su lógica interna.

```
Arboles.h x Arboles.cpp main.cpp
Arboles.h > ArbolBinario > mostrarEstructura()

1  #ifndef ARBOLES_H
2  #define ARBOLES_H
3
4  #include <string>
5
6  struct NodoArbol {
7      int dato;
8      NodoArbol* izq;
9      NodoArbol* der;
10
11      NodoArbol(int valor);
12  };
13
14  class ArbolBinario {
15  private:
16      NodoArbol* raiz;
17
18      NodoArbol* insertarNodo(NodoArbol* nodo, int dato);
19      void preordenRecursivo(NodoArbol* nodo);
20      void inordenRecursivo(NodoArbol* nodo);
21      void postordenRecursivo(NodoArbol* nodo);
22      void mostrarEstructuraRecursivo(NodoArbol* nodo, int espacio);
23
24
25  public:
26      ArbolBinario();
27      void insertar(int dato);
28      void preorden();
29      void inorden();
30      void postorden();
31      void cargarXML(const std::string& nombreArchivo);
32
33      void cargarJSON(const std::string& nombreArchivo);
34      void mostrarEstructura();
35  };
36
37  #endif
```

Arboles.cpp: Aquí Programamos cómo funcionan Cada función declarada en el archivo .h la regla de insertar menores a la izquierda y mayores a la derecha, como se debe de tener un arbol binario, y demás lógicas como para escanear y extraer texto de los archivos (XML y JSON),

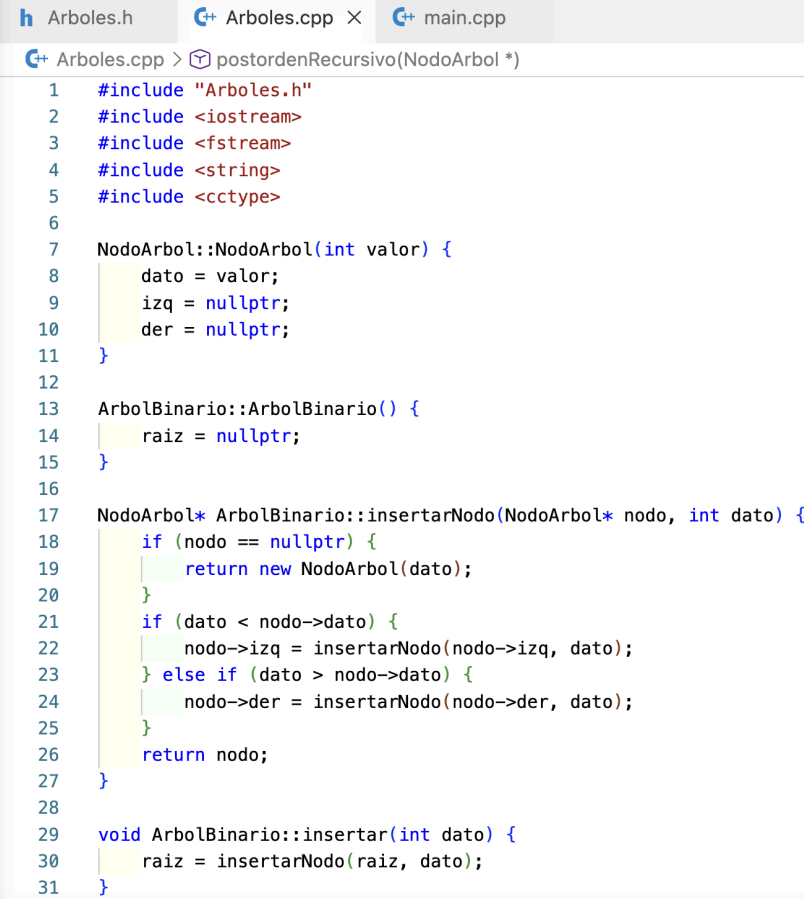
La problemática que resolvimos fue la visualización. Por lo general, un árbol se dibuja de arriba hacia abajo, pero como la consola imprime texto por renglones, adaptamos la estructura a un formato horizontal. De esta manera, la raíz queda pegada a la izquierda, los nodos del subárbol derecho crecen hacia arriba en la pantalla, y los nodos del subárbol izquierdo crecen hacia abajo.

¿Cómo es el recorrido para cada tipo de tipo de ordenamiento?:

void ArbolBinario::preordenRecursivo(NodoArbol* nodo): Comienza visitando la raíz principal, luego atraviesa todo el subárbol izquierdo y finalmente recorre el subárbol derecho.

void ArbolBinario::inordenRecursivo(NodoArbol* nodo): Recorre primero el subárbol izquierdo (menores), pasa por la raíz y termina en el subárbol derecho (mayores). Este es el recorrido que permite mostrar los elementos ordenados estrictamente de menor a mayor.

void ArbolBinario::postordenRecursivo(NodoArbol* nodo): Atraviesa primero el subárbol izquierdo, luego el subárbol derecho y visita la raíz hasta el final. Se procesan primero las hojas antes de llegar a los nodos superiores.



```
1  #include "Arboles.h"
2  #include <iostream>
3  #include <fstream>
4  #include <string>
5  #include <cctype>
6
7  NodoArbol::NodoArbol(int valor) {
8      dato = valor;
9      izq = nullptr;
10     der = nullptr;
11 }
12
13 ArbolBinario::ArbolBinario() {
14     raiz = nullptr;
15 }
16
17 NodoArbol* ArbolBinario::insertarNodo(NodoArbol* nodo, int dato) {
18     if (nodo == nullptr) {
19         return new NodoArbol(dato);
20     }
21     if (dato < nodo->dato) {
22         nodo->izq = insertarNodo(nodo->izq, dato);
23     } else if (dato > nodo->dato) {
24         nodo->der = insertarNodo(nodo->der, dato);
25     }
26     return nodo;
27 }
28
29 void ArbolBinario::insertar(int dato) {
30     raiz = insertarNodo(raiz, dato);
31 }
```

```

32
33 void ArbolBinario::mostrarEstructuraRecursivo(NodoArbol* nodo, int espacio) {
34     if (nodo == nullptr) {
35         return;
36     }
37
38     espacio += 5;
39
40     mostrarEstructuraRecursivo(nodo->der, espacio);
41
42     std::cout << '\n';
43     for (int i = 5; i < espacio; i++) {
44         std::cout << " ";
45     }
46     std::cout << nodo->dato << "\n";
47
48     mostrarEstructuraRecursivo(nodo->izq, espacio);
49 }
50
51 void ArbolBinario::mostrarEstructura() {
52     std::cout << "\n-- Estructura de un arbol binario --\n";
53     mostrarEstructuraRecursivo(raiz, 0);
54 }
55
56
57
58 void ArbolBinario::preordenRecursivo(NodoArbol* nodo) {
59     if (nodo != nullptr) {
60         std::cout << nodo->dato << "-";
61         preordenRecursivo(nodo->izq);
62         preordenRecursivo(nodo->der);
63     }
64 }
65
66 void ArbolBinario::inordenRecursivo(NodoArbol* nodo) {
67     if (nodo != nullptr) {
68         inordenRecursivo(nodo->izq);
69         std::cout << nodo->dato << "-";
70         inordenRecursivo(nodo->der);
71     }
72 }
73
74 void ArbolBinario::postordenRecursivo(NodoArbol* nodo) {
75     if (nodo != nullptr) {
76         postordenRecursivo(nodo->izq);
77         postordenRecursivo(nodo->der);
78         std::cout << nodo->dato << "-";
79     }
80 }
81
82 void ArbolBinario::preorden() {
83     preordenRecursivo(raiz);
84 }
85
86 void ArbolBinario::inorden() {
87     inordenRecursivo(raiz);
88 }

```

```

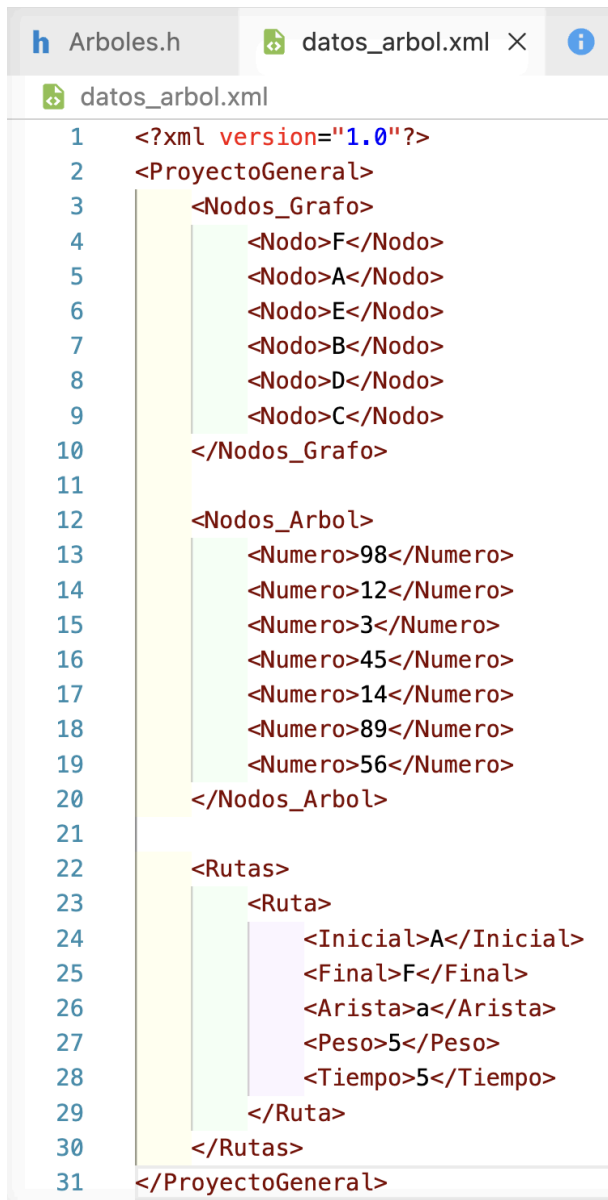
85
86 void ArbolBinario::postorden() {
87     postordenRecursivo(raiz);
88 }
89 void ArbolBinario::cargarXML(const std::string& nombreArchivo) {
90     std::ifstream archivo(nombreArchivo);
91     if (!archivo.is_open()) return;
92
93     std::string linea;
94     while (std::getline(archivo, linea)) {
95         size_t inicio = linea.find("<Numero>");
96         size_t fin = linea.find("</Numero>");
97         if (inicio != std::string::npos && fin != std::string::npos) {
98             inicio += 8;
99             std::string valorStr = linea.substr(inicio, fin - inicio);
100             insertar(std::stoi(valorStr));
101         }
102     }
103     archivo.close();
104 }
105 void ArbolBinario::cargarJSON(const std::string& nombreArchivo) {
106     std::ifstream archivo(nombreArchivo);
107     if (!archivo.is_open()) return;
108
109     char c;
110     std::string numeroStr = "";
111     while (archivo.get(c)) {
112         if (std::isdigit(c) || c == '-') {
113             numeroStr += c;
114         } else {
115             if (!numeroStr.empty()) {
116                 insertar(std::stoi(numeroStr));
117                 numeroStr = "";
118             }
119         }
120     }
121     if (!numeroStr.empty()) {
122         insertar(std::stoi(numeroStr));
123     }
124     archivo.close();
125 }

```

Archivo XML

Es un documento de texto estructurado mediante un sistema de etiquetas jerárquicas (parecido al diseño de una página web). En tu proyecto, este archivo contiene un bloque maestro llamado `<ProyectoGeneral>`, el cual se divide internamente en secciones con etiquetas propias:

- Contiene la etiqueta `<Nodos_Grafo>` y `<Rutas>` donde se guardan las letras.

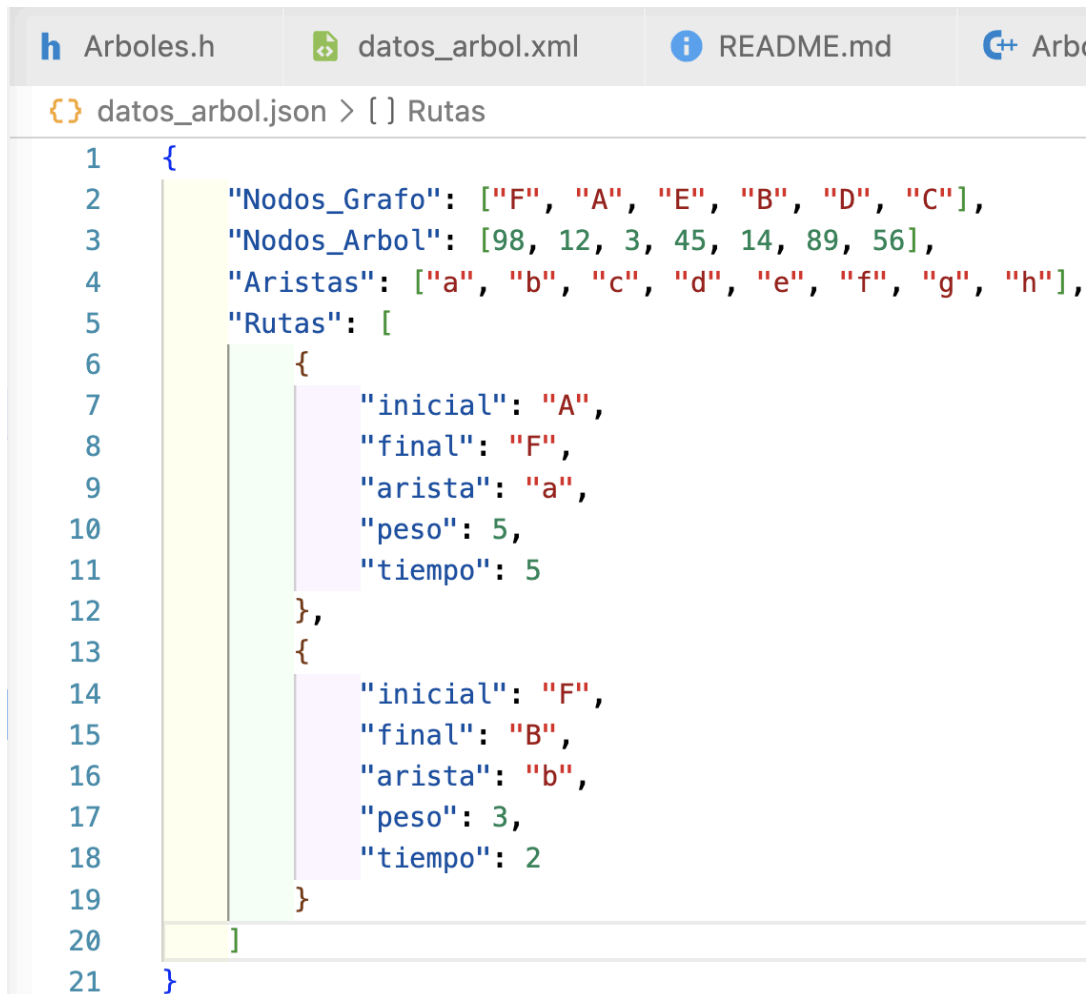


```
1  <?xml version="1.0"?>
2  <ProyectoGeneral>
3      <Nodos_Grafo>
4          <Nodo>F</Nodo>
5          <Nodo>A</Nodo>
6          <Nodo>E</Nodo>
7          <Nodo>B</Nodo>
8          <Nodo>D</Nodo>
9          <Nodo>C</Nodo>
10     </Nodos_Grafo>
11
12     <Nodos_Arbol>
13         <Numero>98</Numero>
14         <Numero>12</Numero>
15         <Numero>3</Numero>
16         <Numero>45</Numero>
17         <Numero>14</Numero>
18         <Numero>89</Numero>
19         <Numero>56</Numero>
20     </Nodos_Arbol>
21
22     <Rutas>
23         <Ruta>
24             <Inicial>A</Inicial>
25             <Final>F</Final>
26             <Arista>a</Arista>
27             <Peso>5</Peso>
28             <Tiempo>5</Tiempo>
29         </Ruta>
30     </Rutas>
31 </ProyectoGeneral>
```

Archivo Json

Es un documento de texto mucho más ligero y moderno que organiza la información usando pares de "llave" y "valor". En lugar de etiquetas repetitivas, agrupa los datos utilizando corchetes [] y llaves { }. En tu proyecto, este archivo contiene:

- Llaves como "Nodos_Grafo" y "Rutas" con arreglos de datos



```
h Arboles.h  datos_arbol.xml  README.md  Arbo
datos_arbol.json > [ ] Rutas
1  {
2      "Nodos_Grafo": ["F", "A", "E", "B", "D", "C"],
3      "Nodos_Arbol": [98, 12, 3, 45, 14, 89, 56],
4      "Aristas": ["a", "b", "c", "d", "e", "f", "g", "h"],
5      "Rutas": [
6          {
7              "inicial": "A",
8              "final": "F",
9              "arista": "a",
10             "peso": 5,
11             "tiempo": 5
12         },
13         {
14             "inicial": "F",
15             "final": "B",
16             "arista": "b",
17             "peso": 3,
18             "tiempo": 2
19         }
20     ]
21 }
```


Main.cpp: Este se encarga de crear los árboles en blanco en la memoria, mandar a llamar a las funciones que extraen los números desde los archivos XML y JSON, y finalmente, dar la orden para que se impriman en la pantalla los recorridos matemáticos (Inorden, Preorden, Postorden) y la estructura visual del árbol.

```
h Arboles.h  C++ Arboles.cpp  C++ main.cpp X
C++ main.cpp > main()
1  #include <iostream>
2  #include "Arboles.h"
3
4  int main() {
5      ArbolBinario arbolXML;
6      arbolXML.cargarXML("datos_arbol.xml");
7
8      std::cout << "---- Arbol en formato XML ----\n";
9      arbolXML.mostrarEstructura();
10
11     std::cout << "\nPreorden: ";
12     arbolXML.preorden();
13     std::cout << "\n\n";
14
15
16     std::cout << "\nInorden: ";
17     arbolXML.inorden();
18     std::cout << "\n\n";
19
20
21     std::cout << "\nPostorden: ";
22     arbolXML.postorden();
23     std::cout << "\n\n";
24
25     ArbolBinario arbolJSON;
26     arbolJSON.cargarJSON("datos_arbol.json");
27
28     std::cout << "---- Arbol en formato JSON ----\n";
29     arbolXML.mostrarEstructura();
30     std::cout << "\n\n";
31
32     std::cout << "Preorden: ";
33     arbolJSON.preorden();
34     std::cout << "\n\n";
35
36     std::cout << "\nInorden: ";
37     arbolJSON.inorden();
38     std::cout << "\n\n";
39
40     std::cout << "\nPostorden: ";
41     arbolJSON.postorden();
42     std::cout << "\n\n";
43
44     return 0;
45 }
```

Programa ejecutado. Tanto en Formato XML como en Json

```
aldomejia@MacBook-Air-de-Aldo parcial3 % g++ main.cpp Arboles.cpp -o programa && ./programa  
--- Arbol en formato XML ---
```

```
-- Estructura de un arbol binario --
```

```
98
```

```
    89
```

```
      56
```

```
    45
```

```
      14
```

```
    12
```

```
      3
```

```
Preorden: 98-12-3-45-14-89-56-
```

```
Inorden:  3-12-14-45-56-89-98-
```

```
Postorden: 3-14-56-89-45-12-98-
```

```
--- Arbol en formato JSON ---
```

```
-- Estructura de un arbol binario --
```

```
98
```

```
    89
```

```
      56
```

```
    45
```

```
      14
```

```
    12
```

```
      3
```

```
Preorden: 98-12-3-2-5-45-14-89-56-
```

```
Inorden:  2-3-5-12-14-45-56-89-98-
```

```
Postorden: 2-5-3-14-56-89-45-12-98-
```